TP6 — Unidad 4: FNBC y Desnormalización Controlada (Food Store)

Informe breve de los dos entregables de script. Base: `food_store_tp3` (PostgreSQL 18), copia poblada (600.000 líneas de `detalle_pedido`) más un día de operación real sembrado (400 pedidos del día).

Entregables: 1. `01_tp_fnbc_control_lote.sql` — Parte 1 (FNBC sobre ControlLoteAlmacen). 2. `02_tp_desnormalizacion_top_categorias.sql` — Parte 2 (reporte top 5 de categorías). 3. `00_setup_ambiente.sql` — preparación idempotente (maestras mayorista, baja lógica en `pedido` / `detalle_pedido`, seed del día). 4. Capturas de EXPLAIN ANALYZE en `evidencia/`.

Parte 1 — FNBC sobre `control_lote_almacen`

a) Dependencias funcionales

De la regla de negocio, en notación formal (`LoteID`, `DepositoID`, `ResponsableControlID`):

- `(LoteID, DepositoID) → ResponsableControlID` — trivial de la clave primaria: un par (lote, depósito interviniente) determina unívocamente al responsable.
- `ResponsableControlID → DepositoID` — dato maestro de dotación: cada responsable pertenece a **un único** depósito, cualquiera sea el lote.

No existen más dependencias no triviales en el esquema dado.

b) Clausuras y claves candidatas

Subconjunto de atributos	Clausura	¿Superclave?
`{LoteID}`	`{LoteID}`	No
`{DepositoID}`	`{DepositoID}`	No
`{ResponsableControlID}`	`{ResponsableControlID, DepositoID}`	No
`{LoteID, DepositoID}`	`{LoteID, DepositoID, ResponsableControlID}`	**Sí (clave)**
`{LoteID, ResponsableControlID}`	`{LoteID, ResponsableControlID, DepositoID}`	**Sí (clave)**
`{DepositoID, ResponsableControlID}`	`{ResponsableControlID, DepositoID}`	No

Claves candidatas: `(LoteID, DepositoID)` y `(LoteID, ResponsableControlID)`.

Atributos primos: los tres (`LoteID`, `DepositoID` y `ResponsableControlID`), porque cada uno aparece en alguna clave candidata. **No primos:** ninguno. Que todos los atributos sean primos es lo que hace que el esquema esté en 3FN pero aun así falle FNBC (ver c).

c) ¿Cumple FNBC?

Un esquema está en FNBC si **toda dependencia funcional no trivial tiene su determinante como superclave**. La DF violatoria es:

- `ResponsableControlID → DepositoID` (no trivial, no implicada por la PK).
- Su determinante `{ResponsableControlID}` **no es superclave**: su clausura es `{ResponsableControlID, DepositoID}` (no contiene a `LoteID`).

Por lo tanto `control_lote_almacen` **no cumple FNBC** (aunque cumple 3FN, porque su parte dependiente `DepositoID` es un atributo primo).

d) Anomalías habilitadas por la instancia `(501,30,801), (502,30,801), (503,31,802)`

- **Inserción:** no se puede registrar a un nuevo responsable (ej. `803`) con su depósito si aún no controla ningún lote, porque `LoteID` es parte de la PK y debe ser no nulo; el dato maestro de `803` queda "colgando" sin poder cargarse.
- **Borrado:** si el lote `503` se anula, se pierde también la única ocurrencia que registra que el responsable `802` pertenece al depósito `31`; borrar los dos lotes de `801` eliminaría la única evidencia de que `801` pertenece al depósito `30`.
- **Actualización:** si `801` cambia de depósito, hay que actualizar **las dos** filas `(501,30)` y `(502,30)`; si se actualiza una sola, el mismo responsable queda con dos depósitos distintos (inconsistencia).

e) Descomposición sin pérdida

Algoritmo: se descompone por la DF violatoria `ResponsableControlID → DepositoID`:

- `R1 lote_responsable(LoteID, ResponsableControlID)` — PK `(LoteID, ResponsableControlID)`.
- `R2 responsable_deposito(ResponsableControlID, DepositoID)` — PK `ResponsableControlID`.

Ambas con FK reales a `lote`, `usuario` y `deposito`, y una vista `v_control_lote_almacen` que reconstruye la relación original por **reunión natural** sobre `ResponsableControlID`. Todo en `01_tp_fnbc_control_lote.sql`.

f) Unión sin pérdida y migración verificada

El atributo común de la descomposición, `ResponsableControlID`, es **superclave de** `R2` (es su clave primaria). Por el criterio de superclave de la clase, la reunión natural `R1 ⋈ R2` reconstruye exactamente la relación original sin tuplas espurias ni pérdida: cada fila de `R1` se combina con **una sola** fila de `R2`.

Migración ejecutada y verificada en el motor:

```
INSERT INTO lote_responsable ... SELECT DISTINCT lote_id, responsable_control_id FROM control_lote_almacen;
INSERT INTO responsable_deposito ... SELECT DISTINCT responsable_control_id, deposito_id FROM control_lote_a
-- 1) en vista y NO en original ........ (0 filas)
-- 2) en original y NO en vista ........ (0 filas)
```

Ambos `EXCEPT` dan **0 filas**: la vista de compatibilidad reproduce la instancia original.

g) Migración transaccional reversible y plan de reversión (objetivo oficial U4, 2ª entrega)

`04_migracion_transaccional_reversible.sql` (especificado en `specs/kiro_fnbc/`) envuelve la migración en transacciones reversibles y fue probado **con corrida real** (evidencia: `evidencia/04_migracion_transaccional_20261109_235808.txt`, 0,68 s) sobre una copia restaurada de un dump verificado (`food_store_tp3_u4_fnbc_20261109_235630`; el original `food_store_tp3` nunca fue destino — el script tiene guarda que lo bloquea):

- **Corrida A (COMMIT verificado):** `BEGIN` → recreación de R1/R2 + vista → `INSERT ... SELECT` desde `control_lote_almacen` (solo lectura) → bloque `DO` que verifica `EXCEPT = 0` en ambos sentidos **dentro de la transacción** → `COMMIT` (última sentencia: solo se alcanza si la verificación pasó). Resultado: `VERIFICACION OK: EXCEPT=0 en ambos sentidos (filas en vista=3)`.

- **Corrida B (ROLLBACK demostrado):** se replica la migración desde una copia **temporal** del origen contaminada con `(501,31,801)`, fila que viola la DF `ResponsableControlID → DepositoID` (801 ya tiene el depósito 30). El esquema FNBC **no tiene dónde guardar el dato inválido**: `responsable_deposito_pkey` rechaza al responsable 801 duplicado (`llave duplicada viola restricción de unicidad «responsable_deposito_pkey»`), la transacción aborta a mitad de migración y el `ROLLBACK` restaura el estado previo exacto — incluido el DDL, transaccional en PostgreSQL. Verificación post-rollback: las 3 tablas y la vista con las mismas filas que antes (`3/3/2/3`) y `control_lote_almacen` con su instancia intacta.
- **Plan de reversión obligatorio** (`specs/kiro_fnbc/design.md` §6): como la migración solo **lee** `control_lote_almacen` (nunca lo borra ni actualiza), revertir tras un `COMMIT` es eliminar los objetos derivados (vista + R1/R2, con opción conservadora de renombrarlas `*_deprecada` para auditoría) sin pérdida de datos; a mitad de migración, basta `ROLLBACK` (demostrado en la Corrida B).

Parte 2 — Desnormalización controlada del panel (top 5 del día)

a) Relevamiento con EXPLAIN ANALYZE

Se midió la consulta del panel. **Corrección previa del filtro de fecha:** el enunciado filtra `ped.fecha = CURRENT_DATE`. Como `fecha` es `TIMESTAMPTZ` y `CURRENT_DATE` es `DATE`, esa igualdad matchea únicamente la medianoche exacta del día (evidenciado con una ejecución que devolvió `actual rows=0` pese a haber 400 pedidos del día). Se corrigió al rango equivalente `ped.fecha >= CURRENT_DATE AND ped.fecha < CURRENT_DATE + 1`, que no cambia el plan ni la temática y sí devuelve el reporte real.

Plan **antes** (extracto de `evidencia/explain_antes.txt`):

```
Limit  (cost=696.27..696.28 rows=4 width=40) (actual rows=4.00 loops=1)
  Buffers: shared hit=5241
  ->  Sort  (cost=695.86..696.23 rows=4 width=40) (actual rows=4.00)
  ->  GroupAggregate  (cost=695.86..696.23)  Group Key: c.nombre
  ->  Nested Loop  (categoria)  ... Rows Removed by Join Filter: 1768
  ->  Nested Loop  (detalle_pedido -> producto)
        ->  Bitmap Heap Scan on pedido (400 filas del dia)
        ->  Index Scan using idx_detalle_pedido (400 loops, 1.608 buffers)
        ->  Index Scan using producto_pkey (1.205 loops, 3.615 buffers)
Planning Time: 56.121 ms
Execution Time: 23.586 ms
```

Tiempo: 23.586 ms. Nodo que domina el costo: la cadena de bucles anidados que, por los 400 pedidos del día, vuelve a recorrer `detalle_pedido` (400 `Index Scan`, 1.608 buffers) y resuelve la unión con `producto` fila por fila (1.205 `Index Scan` sobre `producto_pkey`, **3.615 buffers**), para recién después agregar y ordenar. Son 1.205 filas de detalle procesadas con 5.241 buffers tocados **en cada ejecución** del panel: el costo crece a medida que crecen los pedidos del día sobre una tabla de 600.000 líneas.

b) Patrón elegido y justificación

Se eligió vista materializada (`mv_top_categorias_dia`) y no columna precalculada con disparador.

- **¿Qué evidencia mide la decisión?** En (a) el 65% de los buffers se concentra en la unión con `producto` y en el barrido de `detalle_pedido` por pedido (nodos `Nested Loop` + `Index Scan`), trabajo que el panel repite

en cada refresco (muchas veces por minuto). Precalcular el agregado por `(dia, categoria)` lo elimina por completo de la lectura.

- **¿Qué mecanismo evita la desincronización?** la MV es una proyección 100 % derivada de la fuente de verdad (las 4 tablas), con el filtro de baja lógica embebido en su definición; se re-materializa con `REFRESH MATERIALIZED VIEW CONCURRENTLY` (habilitado por su índice único), programado por el panel, sin bloquear lecturas. El dato redundante no se escribe "a mano": solo puede quedar atrasado, y la auditoría (e) lo detecta.
- **¿Es reversible sin pérdida?** Sí: `DROP MATERIALIZED VIEW` elimina el objeto redundante sin tocar las tablas de origen, que siguen siendo la única verdad. Comparado con el trigger, la MV evita mantener sumas en cada `INSERT/UPDATE/DELETE` de `detalle_pedido` (alta contención de escritura y lógica cruzada a `pedido` / `producto`), y traslada el costo a un refresco único y amortizable.

c) Implementación

`02_tp_desnormalizacion_top_categorias.sql` crea la MV (agregado por día y categoría), su índice único y ejecuta el `REFRESH CONCURRENTLY` (mecanismo de sincronización).

d) Consulta desnormalizada y comparación

Consulta contra la MV (mismo reporte, sin unir las 4 tablas):

```
SELECT categoria, total_vendido
FROM mv_top_categorias_dia
WHERE dia = CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
```

Plan **después** (extracto de `evidencia/explain_despues.txt`):

```
Limit  (cost=14.00..14.01 rows=4 width=16) (actual rows=4.00 loops=1)
  Buffers: shared hit=9
  ->  Sort  (cost=14.00..14.01 rows=4 width=16) (actual rows=4.00)
  ->  Bitmap Heap Scan on mv_top_categorias_dia  (4 filas)
        ->  Bitmap Index Scan on uq_mv_top_categorias_dia (Index Cond: dia = CURRENT_DATE)
Planning Time: 2.236 ms
Execution Time: 0.134 ms
```

Métrica	Antes (4 tablas)	Después (MV)
Tiempo de ejecución	**23.586 ms**	**0.134 ms** (~176× más rápido)
Buffers tocados	5.241	9
Nodo dominante	`Nested Loop` + `Index Scan` por fila (detalle 400×, producto 1.205×)	`Bitmap Index Scan` sobre `uq_mv_top_categorias_dia` (4 filas)

El resultado del reporte (top 4 de hoy: Pizzas, Bebidas, Empanadas, Postres) es idéntico en ambos planes; la MV devuelve el agregado ya calculado leyendo solo 4 filas.

e) Auditoría de desincronización

Script incluido en `02_tp_desnormalizacion_top_categorias.sql`: recalcula el agregado completo desde las 4 tablas y lo compara fila por fila contra la MV con `FULL JOIN` + `IS DISTINCT FROM`. Sobre la base ya migrada devuelve:

```
 dia | categoria_id | mv_total | fuente_total | tipo_desync
-----+--------------+----------+--------------+-------------
(0 filas)
```

Resultado vacío: la MV está sincronizada con la fuente de verdad.